Activity: Compare Pruned vs Full CNN Performance

Introduction

In this activity, you will explore model compression through pruning, a technique that reduces the size of deep neural networks by setting a portion of their weights to zero. By training a baseline Convolutional Neural Network (CNN) on the CIFAR-10 image classification dataset, you will observe how pruning can help compress the model while aiming to retain comparable performance. You'll implement structured steps to train a full model, apply weight pruning using PyTorch's pruning utilities, fine-tune the compressed model, and analyze the differences in accuracy, inference speed, and sparsity between the original and pruned networks.

Learning Outcome

- Train a CNN on the CIFAR-10 dataset and apply pruning to reduce its size.
- Compare the inference speed and accuracy of the original vs. pruned model and analyze the impact of compression.

How to Run Your `.ipynb` File on Colab (Recommended)

Option 1: Upload Directly

1. Go to https://colab.research.google.com
2. Click **"Upload"** tab → Select your `.ipynb` file from your computer
3. Colab will open it in a new tab
4. Make sure to go to `Runtime → Change runtime type` and set:
    - **Hardware accelerator:** `GPU`

Option 2: Mount from Google Drive

If your notebook is stored in Drive:

1. Open Colab
2. Choose **"Google Drive"** tab
3. Navigate to the `.ipynb` file and open it

Before Running Any Cell

Add a **setup cell at the top** of your notebook to ensure dependencies are installed:

```
!pip install -q torch torchvision transformers datasets matplotlib numpy timm datasets
```

Then Run Cell-by-Cell

1. Start with installation
2. Set up device check and conditional DataLoader
3. Proceed through data, model, training, and evaluation

Optional: Save Outputs Back to Google Drive

If you want to save trained model or results:

```
from google.colab import drive
drive.mount('/content/drive')

# Example to save
torch.save(model_vit.state_dict(), '/content/drive/MyDrive/vit_cifar10.pth')
```

Otherwise if you are using you machine follow the follwing Steps:

Working with Dev Container

To complete this assignment in a reliable and fully configured environment, please refer to the instructions in the file: `README-devcontainer.md`. This guide walks you through opening the assignment in **Visual Studio Code** using a **Dev Container**, which automatically installs all necessary Python libraries and tools. Following that setup ensures that the notebook runs smoothly without manual configuration or missing dependencies. Make sure to open the **main activity folder** in VS Code and follow the steps outlined in the Dev Container guide before starting the notebook.

Starter File

- Open the notebook: `activity_compare_pruned_full_cnn_performance.ipynb` in the `Code` folder.
- Run it locally or in **Google Colab** with a GPU accelerator.
- Ensure the following libraries are installed: `torch`, `torchvision`, `numpy`.

Code Steps

1. **Load and Preprocess Data**: Load the CIFAR-10 dataset using `torchvision.datasets.CIFAR10` and apply a transformation to convert images to tensors.
2. **Train Full Model**:
3. **Evaluate Full Model**: Measure the accuracy and inference time of the full model on the test set.
4. **Apply Pruning**: Use `torch.nn.utils.prune.l1_unstructured` to prune 50% of the weights in both convolutional layers based on L1 norm.
5. **Evaluate Pruned Model**: Measure the accuracy and inference time of the pruned model before fine-

tuning.

6. **Fine-Tune Pruned Model**: Fine-tune the pruned model for 5 epochs with a lower learning rate (1e-4) using the Adam optimizer.
7. **Evaluate Fine-Tuned Model**: Measure the accuracy and inference time of the fine-tuned pruned model.
8. **Analyze Compression**: Calculate the sparsity of the convolutional layers in the pruned model.

Conclusion

In this activity, you successfully trained a CNN on the CIFAR-10 dataset and applied pruning to reduce its size by eliminating 50% of the weights in the convolutional layers. By comparing the inference time, accuracy, and sparsity of the full and pruned models, you gained insights into how model pruning can reduce complexity while improving performance after fine-tuning.
now, if you'd like me to generate a chart to visualize the results, or if you need further modifications!